SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q7 – Design an Election Counting Program from First Principles

Stage 1 – Interpret the Problem (Joint)

- **Input:** c ≤ 12 candidates; up to V = 5,000 integer votes. Rule: a vote x is valid only if 1 ≤ x ≤ c; anything else (0, negative, above c) is invalid.
- **Output:** number of valid and invalid votes; the votes for each candidate in descending order; the winner; whether there is a tie.
- **Arrays:** votes[5000] (20,000 bytes; not needed if votes are processed as they arrive), count[13] (indices 1..12; index 0 unused), results[12] of (candidate, votes) pairs.

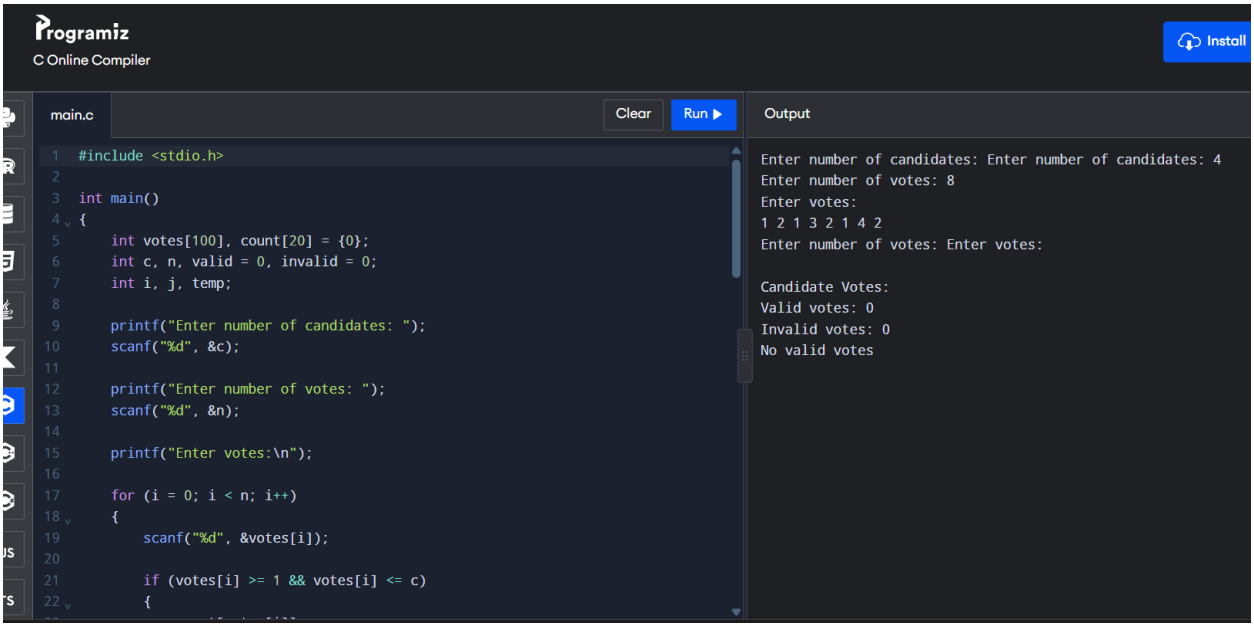
Measurable requirements:

- R1 valid + invalid = number of votes entered.
- R2 the sum of all candidate counts = valid votes.
- R3 the printed list is in non-increasing order of votes.
- R4 the winner's count is the maximum; a tie is reported exactly when two or more candidates share the maximum.
- R5 one pass over the votes: time O(V + c log c), with c ≤ 12 and V ≤ 5,000 taking well under a millisecond (measured below).

Stage 2 – Design A

```
COUNT(votes, V, c):
   count[1..c] = 0 ; valid = 0 ; invalid = 0
   for each vote x in votes:
      if 1 <= x <= c: count[x] = count[x] + 1 ; valid = valid + 1
      else:           invalid = invalid + 1
   build results[i] = (i, count[i]) for i = 1..c
   sort results by votes descending (equal votes: lower candidate number first)
   if valid == 0: report "no valid votes"
   tied = 1
   while tied < c and results[tied].votes == results[0].votes: tied = tied + 1
   if tied > 1: report "TIE between", results[0..tied-1]
   else:        report "winner", results[0]
```

Tie detection: after sorting, all candidates with the maximum are at the front, so a tie exists exactly when the second candidate has the same count as the first; counting how many share it names the tied candidates. Sorting 12 results costs a negligible O(c log c) (any simple sort will do).



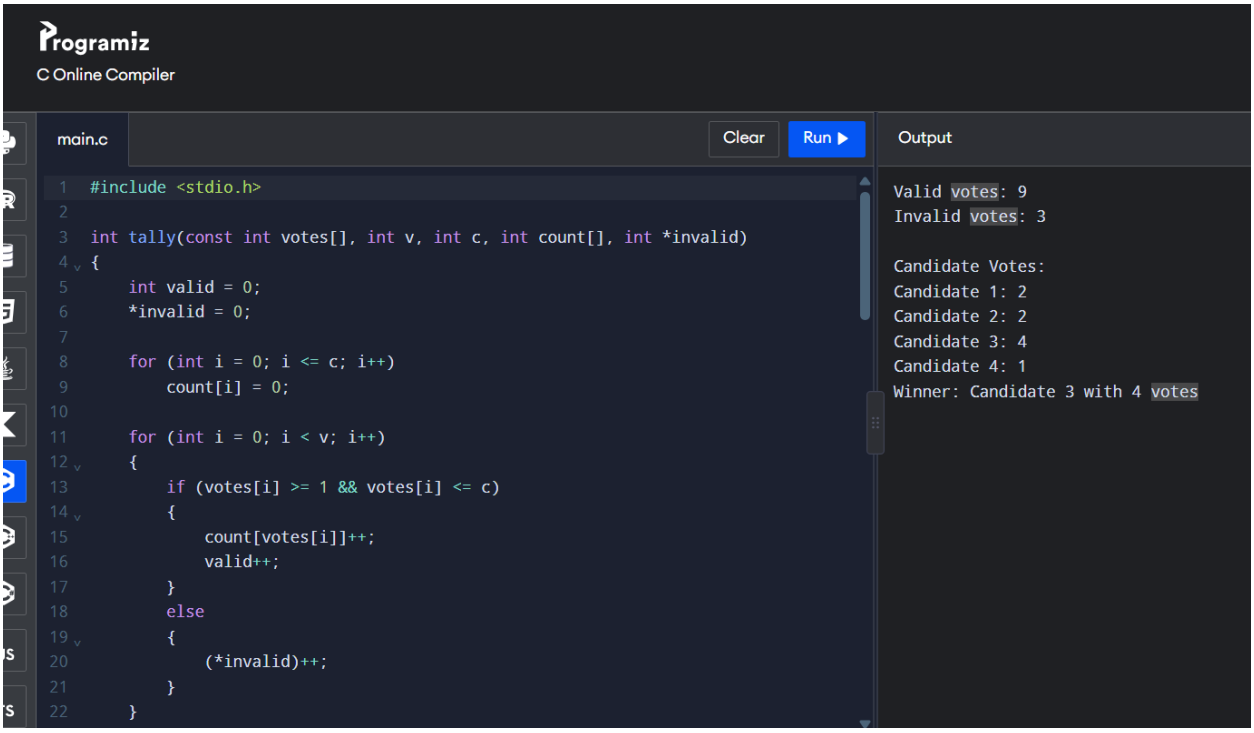
Stage 3 – Implement and Measure (B)

```
int tally(const int votes[], int v, int c, int count[], int *invalid)
{
   int valid = 0; *invalid = 0;
   for (int i = 0; i <= c; i++) count[i] = 0;
   for (int i = 0; i < v; i++) {
      if (votes[i] >= 1 && votes[i] <= c) { count[votes[i]]++; valid++; }
      else (*invalid)++;
   }
   return valid;
}   /* results sorted with qsort and a comparator: more votes first, then lower number */
```

Test	Votes	Output	Hand check
Invalid votes (c = 4)	1 2 2 3 0 3 3 4 −2 5 1 3	valid = 9, invalid = 3; cand 3: 4, cand 1: 2, cand 2: 2, cand 4: 1; winner 3	3 appears 4 times; 0, −2, 5 invalid ✓
Tie (c = 2)	1 2 1 2 9	valid = 4, invalid = 1; TIE between 1 and 2 with 2 votes	✓
All invalid (c = 3)	0 −1 7	valid = 0, invalid = 3; "no valid votes"	✓

Votes	Count array (measured)	Search a name list for every vote	Sort all votes, count runs
5,000	0.012 ms	0.444 ms (28,140 strcmp)	0.421 ms
500,000	2.88 ms	51.9 ms (2,785,180 strcmp)	52.3 ms

Times are gcc -O0 on my machine (averaged over repeated runs; yours will differ). The count array took 2.4 ns per vote at 5,000 votes and 5.8 ns per vote at 500,000 – close to linear; the small increase comes from memory effects. For 500,000 votes the count array was about 18 times faster than the name search and the sort-based method.



Stage 4 – Defend the Design (Joint)

- **Why a count array:** the candidate number is itself the index, so each vote is counted in one step, O(1), with no searching. Searching a list of names for every vote costs up to c string comparisons per vote (O(v·c); 2.8 million strcmp calls for 500,000 votes) and needs string comparison instead of a single integer test.
- **For 500 candidates:** count[501] still works unchanged – counting remains O(V) and memory is only 2 KB; the sort of results grows to O(c log c) with c = 500, still trivial; the name search would be 40 times slower. The report would probably show only the top few candidates plus the tie check, and MAXC and the result arrays would be enlarged or allocated dynamically. If candidate numbers were not small consecutive integers (e.g. 6-digit IDs), a hash table or a mapping from ID to index would be needed.
- **A remaining limitation:** the program trusts that one entry is one voter. It cannot detect duplicate voting, does not record why a vote was invalid, and relies on a fixed upper limit on candidates and dense numeric candidate IDs.

Alternative Concept and Complexity Analysis

Method	Time (v votes, c candidates)	Space	Measured at v = 500,000, c = 12
Count array indexed by candidate	O(v + c)	O(c) (plus O(v) only if the votes are stored)	2.88 ms
Search a list of names for each vote	O(v · c)	O(c)	51.9 ms
Sort all votes, count runs of equal values	O(v log v)	O(v) to hold the votes	52.3 ms

The count array is best whenever the categories are a small, dense range of integers. The name search is the natural way when only names are available but scales badly with c; sorting is attractive only when the categories are unbounded or not numeric.

Feedback and Reflection;

This assignment was designed as a group project, but I completed it individually. Doing so gave me full ownership of every task, from diagnosing the poor data design to implementing the canteen system, and deepened my understanding of structures, unions, storage classes and memory layout. The main limitation was the increased workload and the absence of peer review, which a group would have provided. In future, I would welcome a group version to benefit from shared ideas and code reviews.